

PROGRESS REPORT

Agentic AI System for Automotive Spare Parts Management

Document Agent — Design and Progress

Prepared by	Mohnish Bhandarkar — Final Year B.Tech
Status	Document Agent complete Vision Agent next
Date	July 2026

1. Abstract

This report covers the **design and progress of the Document Agent**, the first completed module of a multi-agent AI system for automotive spare parts management. The full system is meant to take a request like “I need all parts for the steering assembly of a Mahindra Thar” and handle it end-to-end: read the catalogue, extract the parts list, check stock, and generate a report.

The Document Agent handles the first step: **turning a PDF spare-parts catalogue into a clean, structured Bill of Materials (BOM), saved as a JSON file**. It is built as an eight-step LangGraph workflow that combines PDF parsing and page search with an LLM for the final extraction, validated with Pydantic before being saved.

The agent has been tested on four real catalogues: Apollo Tractors (wheel drive), Mahindra Scorpio (hand brake), Mahindra Thar (steering gear and mountings), and Tata Indica (intake and exhaust valve). This report explains the architecture, each step of the workflow, the retrieval logic, the prompt design, the output schema, testing, results, limitations, and the plan for the next milestone — the Vision Agent.

2. Problem Statement

Service centres and dealerships regularly request spare parts for repairs. A spare parts factory holds inventory across hundreds of assemblies, but today, matching a request to the right parts is mostly manual.

Assembly information lives inside PDF catalogues, not in any structured database. When a request comes in, someone has to open the right catalogue, find the right section (which can span several pages and sit next to unrelated assemblies), read a table, and manually check it against inventory. This is slow and does not scale as more catalogues and vehicle models are added.

This project asks: can an AI system do this extraction and matching automatically, without a person manually reading every catalogue page?

3. Objectives

The overall project builds a multi-agent system that reads catalogues, understands exploded-view diagrams, checks a mock inventory database, flags shortages, and produces a report. The **objectives** covered by this report, for the **Document Agent**, are:

- Load and index any PDF spare-parts catalogue.
- Find the pages relevant to a plain-language assembly query, without page numbers being given.
- Extract both text and tables from the relevant pages.
- Convert the extracted content into a clean, structured BOM using an LLM guided by a clear prompt.
- Validate the output against a schema before saving it.
- Save the result as a predictable JSON file that later agents can use.
- Test the approach across catalogues from different manufacturers with different layouts.

4. Overall Multi-Agent Architecture

The full system is designed as a pipeline of five agents, each with one job. This is similar to how a team would split the same task: one person reads the brochure, one reads the diagram, one checks the warehouse, one decides what is short, and one writes the report.

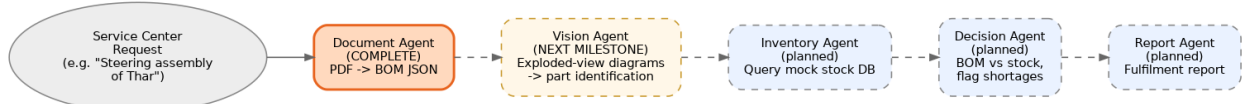


Figure 1: Five-agent pipeline. The Document Agent is complete; the Vision Agent is the next milestone.

Agent	Responsibility	Status
Document Agent	Parse PDF catalogues and extract a structured BOM for the requested assembly	Complete (this report)
Vision Agent	Read exploded-view diagrams using a multimodal model to identify parts and part numbers	Next milestone
Inventory Agent	Query a mock inventory database and return stock levels for each part	Planned
Decision Agent	Compare the BOM against stock and flag parts that are unavailable or low	Planned
Report Agent	Produce a readable fulfilment report for the service centre	Planned

Since only the Document Agent is built and tested so far, the rest of this report focuses on it. Its output (a validated BOM JSON file) is what the Inventory Agent will read once it is built.

5. LangGraph Workflow — Document Agent

The Document Agent is built as a **linear StateGraph** using **LangGraph**. LangGraph was chosen because it keeps the pipeline's state explicit at every step and lets the graph be visualised directly from the code, without drawing it by hand.

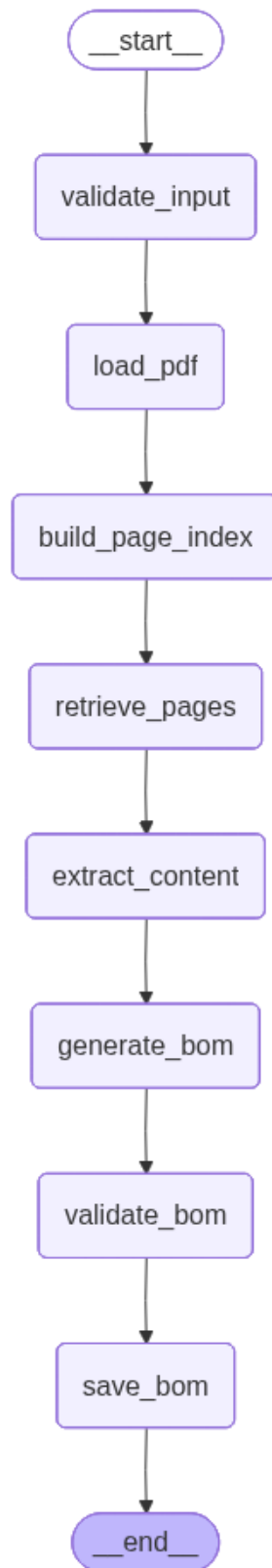


Figure 2: Workflow graph generated directly from the running code (`agents.document.main`), showing the eight steps in order.

The graph is defined in `graph.py` by registering eight nodes on a `StateGraph` typed to `DocumentState`, then connecting them in order, starting at `validate_input` and ending at `save_bom`. Each node receives the shared state, updates the relevant fields, and passes it to the next node.

```

workflow = StateGraph(DocumentState)
workflow.add_node("validate_input", validate_input)
workflow.add_node("load_pdf", load_pdf)
workflow.add_node("build_page_index", build_page_index)
workflow.add_node("retrieve_pages", retrieve_relevant_pages)
workflow.add_node("extract_content", extract_relevant_content)
workflow.add_node("generate_bom", generate_bom)
workflow.add_node("validate_bom", validate_bom)
workflow.add_node("save_bom", save_bom)
workflow.set_entry_point("validate_input")
# edges chain each node to the next, ending at END

```

The DocumentAgent class (agent.py) builds this graph once, and exposes a single invoke(pdf_path, query) method. It sets up the starting DocumentState with empty values for every field the graph will fill in, so each node can assume every key already exists.

6. DocumentState — The Shared State

DocumentState (state.py) is a **TypedDict** that defines the single object passed through every node. Using a typed dictionary rather than a plain dict makes the contract between nodes clear and easy to follow.

Field	Type	Set By	Purpose
pdf_path	str	Caller (invoke)	File path of the source catalogue
user_query	str	Caller (invoke)	Plain-language assembly request, e.g. "WHEEL DRIVE"
assembly_name	str	Reserved	Reserved for future use
pdf_document	Any	load_pdf	In-memory PyMuPDF document handle
page_index	Dict[int, str]	build_page_index	Page number → extracted text
relevant_pages	List[int]	retrieve_relevant_pages	Pages selected as relevant to the query
extracted_text	str	extract_relevant_content	Combined text of the relevant pages
extracted_tables	List[Dict]	extract_relevant_content	Tables found by pdfplumber
bom	Dict[str, Any]	generate_bom / validate_bom	Structured BOM, before and after validation
output_file	str	save_bom	Path of the saved BOM JSON file
status	str	Every node	Current pipeline stage, used for tracking progress
error	str	Any node on failure	Error message if a step fails

The status field acts as a simple trace: it is updated at the end of every node ("validated", "pdf_loaded", "page_index_created", "pages_retrieved", "content_extracted", "bom_generated", "completed", "saved"), so the exact stage reached can always be read from the final state. This is useful for spotting where a run failed.

7. Step-by-Step Explanation

7.1 validate_input

The first step checks three things before any real work starts: that the given `pdf_path` exists, that it ends in `.pdf`, and that the `user_query` is not empty. If any check fails, status is set to "failed" with a clear error message. This keeps every later step free of repeated checks.

7.2 load_pdf

This step opens the catalogue using PyMuPDF (imported as `fitz`) and stores the document handle in `state["pdf_document"]`. PyMuPDF was chosen because it is fast at reading pages in bulk, which matters for catalogues with hundreds of pages. Any failure to open the file is caught and reported.

7.3 build_page_index

Once the document is open, this step goes through every page once, extracts its text with PyMuPDF, and builds a dictionary mapping page number to text — `state["page_index"]`. Doing this once up front means the next step can search the whole catalogue in memory instead of re-reading the file for every page.

7.4 retrieve_relevant_pages

This step finds which pages match the query, since catalogues do not have a machine-readable index. It uses three stages, moving to the next one only if the previous stage finds nothing:

- Exact phrase match — the query is searched for as a literal phrase on each page. Pages under 100 characters are skipped (covers, blank pages). If more than one page matches, the lowest page number is dropped, since it is usually the catalogue's contents page rather than the actual section.
- Keyword scoring — if no exact match is found, each page is scored: +20 if the full query appears anywhere, plus 5 points per occurrence of each query word. The top 5 pages by score are used. This handles cases where the exact phrase is broken up or worded slightly differently.
- Fuzzy matching — as a last resort, each query word is compared against the words on each page using a similarity check (`diffli`, cutoff 0.80), to catch minor spelling or scanning differences. The top 5 pages are used.

If none of the three stages find a match, the step sets status to "failed" with a clear "No pages found" error, rather than returning an empty or wrong result. This makes failures visible instead of silent.

7.5 extract_relevant_content

With the relevant pages known, this step reopens the PDF using `pdfplumber`, which is better at reading table structure than PyMuPDF. For each relevant page it pulls out plain text and any tables, combining the text and collecting all table rows into `state["extracted_tables"]`. Using PyMuPDF for fast full-document indexing and `pdfplumber` for accurate tables on a small set of pages keeps the pipeline both fast and accurate.

7.6 generate_bom

This is where the extracted content is handed to the LLM. The text is first cleaned by `clean_text()`, which removes stray characters (pipes, asterisks, extra parentheses), collapses repeated spaces, and expands a small set of known abbreviations ("ASSY" → "ASSEMBLY", "COLUM" → "COLUMN", "TIE RODEND" → "TIE ROD END"). The cleaned text, the extracted tables, and the user's query are combined with `DOCUMENT_BOM_PROMPT` (Section 9) and sent to the LLM in one call. The reply is parsed directly as JSON into `state["bom"]`. The LLM client is set up once in `llm.py` using LangChain's `ChatOpenAI` wrapper, with `temperature=0` to keep results consistent across runs.

7.7 validate_bom

The JSON from the LLM is not trusted blindly. This step first converts every part's item field to a string (the LLM can return item numbers as either text or numbers, especially for values like “2-1”), then checks the whole result against the BOM schema using Pydantic (Section 10). If a required field is missing or the wrong type, Pydantic raises an error immediately, turning a hidden data problem into a clear, early failure. On success, status is set to “completed”.

7.8 save_bom

The final step saves the validated BOM as a JSON file in an output/ folder (created if it doesn't exist). The filename combines the catalogue name and the query, for example APOLLO_TRACTORS_wheel_drive.json, so repeated runs are easy to find. The file is saved with indent=4 for readability, and state[“output_file”] records the path. This JSON file is the final deliverable of the Document Agent and is what the Inventory Agent will read next.

8. Retrieval Logic — Why This Approach

Page retrieval was built as a cascade of increasingly loose text-matching stages rather than a single semantic search, for two reasons. First, catalogue section titles are usually printed almost exactly as a technician would type them, so exact and keyword matching alone resolves most real queries, with no extra setup cost. Second, a semantic search is more likely to pick a similar-sounding but wrong section (for example “FRONT AXLE” vs “REAR AXLE”), while literal matching fails clearly instead of quietly returning the wrong assembly.

The fuzzy-matching fallback exists to handle small text variations that plain matching would miss. A similarity cutoff of 0.80 was chosen as a balance: tight enough to avoid matching unrelated words, loose enough to still catch small differences.

TOP_K (set to 5) limits how many pages move on to extraction and the LLM step, which keeps the cost and speed of generate_bom predictable regardless of catalogue size.

9. Prompt Design — DOCUMENT_BOM_PROMPT

The extraction prompt (prompts.py) has the biggest effect on output quality, since it is the main way the LLM's behaviour is controlled. It is written as a list of clear rules rather than a short instruction, because early tests showed that short prompts led to merged rows, renumbered items, or missing part numbers. The rules are grouped as follows:

- General rules — use only the supplied content, never guess missing information, return JSON only, no extra text.
- Table rules — one row is one part; never merge or split rows; keep row order; extract only the assembly matching the query, and stop as soon as a different assembly title appears.
- Item number rules — copy item numbers exactly as printed, including values like “2-1” or “11-2”, without renumbering.
- Part number rules — copy part numbers exactly; never guess missing digits.
- Description rules — clean up stray characters and extra spaces, expand a fixed set of abbreviations (ASSY, COL, LH, RH), but do not change real technical terms.
- Column-handling rules — do not add extra column values (like effectivity codes) into the description.
- Quantity rules — quantity must always be a whole number; if unreadable, use 0; codes such as “RF” go into remarks, not into quantity.

- Remarks rules — move qualifiers such as TYPE1/TYPE2, W/ABS, N/ABS, LH, RH, RF into remarks, not into the description.
- Ignore rules — skip page numbers, headers, footers, watermarks, figure borders, company names, and repeated titles.

The prompt ends with the exact JSON shape expected, matching the Pydantic models in Section 10, which helps the LLM stick to the right field names and types. `temperature=0` is used on the LLM call so results stay consistent across repeated runs.

10. Pydantic Schemas

`schemas.py` defines two models that the `validate_bom` step checks the output against. Pydantic gives clear validation errors and one place where the output shape is defined, which both the prompt and later agents can be built around.

```
class BOMItem(BaseModel):
    item: str          # Item number shown in catalogue
    part_number: str    # Manufacturer part number
    description: str     # Part description
    quantity: int = 0    # Quantity required
    remarks: str = ""    # Additional notes if available

class BOM(BaseModel):
    assembly: str
    catalogue: str
    total_parts: int
    parts: List[BOMItem]
```

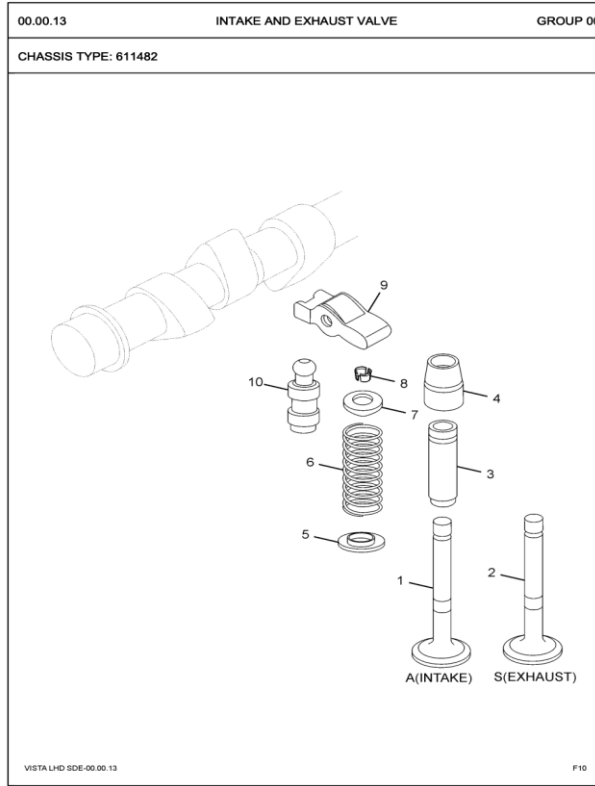
Every `BOMItem` requires `item`, `part_number`, and `description`, since a row missing any of these is not usable. `quantity` and `remarks` default to 0 and an empty string, matching how the prompt handles unreadable or missing values. The outer `BOM` model adds the assembly name, catalogue name, and total part count, so later agents know exactly which assembly and catalogue a BOM came from.

11. Example of a Successful Extraction

This section walks through one full run of the Document Agent, from source catalogue to saved JSON output, using the Tata Indica “Intake and Exhaust Valve” assembly as the example.

11.1 Source catalogue page

The original catalogue page shows the exploded-view diagram for the assembly, with each part numbered 1 to 10.

TATA MOTORS


24

Figure 3: Source exploded-view diagram — Tata Indica, Intake and Exhaust Valve (chassis type 611482).

11.2 Source catalogue table

The same page includes the printed parts table that the Document Agent reads and extracts from.



00.00.13 INTAKE AND EXHAUST VALVE GROUP 00				
CHASSIS TYPE: 611482				
FIG. NO.	PART NUMBER	DESCRIPTION	QTY	REMARKS
1	570305129907	VALVE(INLET)	8	
2	570305129908	VALVE(EXHAUST)	8	
3	570305159905	VALVE GUIDE +0.05	16	
3	570305159906	VALVE GUIDE +0.10	16	
3	570305159907	VALVE GUIDE +0.20	16	
4	570305159908	SEAL	16	
5	570305129909	CUP(LOWER)	16	
6	570305129910	SPRING	16	
7	570305129911	CUP(UPPER)	16	
8	570305129906	PROT.SEMI CONE VALVE	32	
9	570305149903	ROCKER	16	
10	570305149904	TAPPET(HYD. PUSH ROD)	16	

Figure 4: Source parts table for the Intake and Exhaust Valve assembly.

11.3 Agent run and output

Running the agent (python -m agents.document.main) against this catalogue with the query “intake and exhaust valve” produces an exact page match, then extracts and saves the BOM as JSON.

```

(.venv) mohnish@mohnishs-MacBook-Air PERSISTENT_AI % python -m agents.document.main
Workflow graph saved as document_graph_2.png
Exact Match: [36, 37, 92, 93]
BOM saved to: output/TATA_INDICA_intake_and_exhaust_valve.json

Assembly : INTAKE AND EXHAUST VALVE
Catalogue: 661111448822
Total Parts: 12

+-----+-----+-----+-----+-----+
| Item | Part Number | Description | Qty | Remarks |
+-----+-----+-----+-----+-----+
| 1 | 570305129907 | VALVE INLET | 8 | |
+-----+-----+-----+-----+-----+
| 2 | 570305129908 | VALVE EXHAUST | 8 | |
+-----+-----+-----+-----+-----+
| 3 | 570305159905 | VALVE GUIDE +0.05 | 16 | |
+-----+-----+-----+-----+-----+
| 3 | 570305159906 | VALVE GUIDE +0.10 | 16 | |
+-----+-----+-----+-----+-----+
| 3 | 570305159907 | VALVE GUIDE +0.20 | 16 | |
+-----+-----+-----+-----+-----+
| 4 | 570305159908 | SEAL | 16 | |
+-----+-----+-----+-----+-----+
| 5 | 570305129909 | CUP LOWER | 16 | |
+-----+-----+-----+-----+-----+
| 6 | 570305129910 | SPRING | 16 | |
+-----+-----+-----+-----+-----+
| 7 | 570305129911 | CUP UPPER | 16 | |
+-----+-----+-----+-----+-----+
| 8 | 570305129906 | PROT.SEMI CONE VALVE | 32 | |
+-----+-----+-----+-----+-----+
| 9 | 570305149903 | ROCKER | 16 | |
+-----+-----+-----+-----+-----+
| 10 | 570305149904 | TAPPET HYD. PUSH ROD | 16 | |
+-----+-----+-----+-----+-----+

```

Figure 5: Terminal output of a full run — page match, saved file path, and the extracted BOM printed as a table.

The agent correctly matched pages [36, 37, 92, 93], saved the result to output/TATA_INDICA_intake_and_exhaust_valve.json, and reproduced all 12 rows from the source table, including the repeated item number “3” for the three Valve Guide size variants (+0.05, +0.10, +0.20), exactly as printed in the catalogue rather than being renumbered.

12. Catalogue Selection & Sample Outputs

Four real manufacturer catalogues were used for testing, chosen because they differ in layout, section density, and part-number style.

Catalogue	Sample Assembly Queried	Why Selected
Apollo Tractors	Wheel Drive	Clean, simple table layout — a good baseline case.
Mahindra Scorpio	Hand Brake (Group 2201A)	Index page shares space with multiple unrelated assemblies (wheel brake, instrument panel) — tests the “stop at next assembly” rule.
Mahindra Thar	Steering Gear and Mountings (Group 2101)	Has sub-item quantities inside description text and RF/NS remark codes — tests quantity and remarks handling.

Catalogue	Sample Assembly Queried	Why Selected
Tata Indica	Intake and Exhaust Valve	Has a repeated item number (three “VALVE GUIDE” variants under item 3) — tests the “never renumber” rule.

This mix covers a tractor OEM, two Mahindra vehicles, and a Tata passenger car, so the rules tested here are not tuned to just one manufacturer's catalogue style.

12.1 Apollo Tractors — Wheel Drive

A clean 24-part extraction with sequential item numbers 1–24, consistent quantities, and no remarks needed.

Item	Part Number	Description	Qty
1	M10130AA	Wheel Hub Rear-Big Double Tire	4
2	A55150AA	Tyre	4
9	D20500AA	Chain Sprocket	4
13	D10160AA	Chain	4
24	N55235AA	Spring Washer 12 mm	48

Table excerpt: 5 of 24 extracted rows shown. Saved as: output/APOLLO_TRACTORS_wheel_drive.json

12.2 Mahindra Scorpio — Hand Brake (Group 2201A)

Extraction correctly stopped at the boundary of Group 2201A and did not pull in the neighbouring “Group 2202B — Wheel Brake” or “Group 2606A — Instrument Panel” sections, producing exactly 16 parts.

Item	Part Number	Description	Qty
1	0605AA0110N	Lever Parking Brake	1
7	0605BA0120N	Assembly Frt Cable & Conduit HS	1
14	56060A0010N	CBI SCR With SPL Washer M6 x 1 - 12	6
16	0068153	Clip Wire	2

Table excerpt: 4 of 16 extracted rows shown. Saved as: output/MAHINDRA_SCORPIO_hand_brake.json

12.3 Mahindra Thar — Steering Gear and Mountings (Group 2101)

The most complex case tested: item 8 (“Anti Squeak Steering Column”) had an unreadable quantity, correctly defaulted to 0 with the raw fragment kept in remarks instead of being guessed; item 19 correctly captured the “NS” (not supplied) code in remarks with quantity 0.

Item	Part Number	Description	Qty	Remarks
1	1106AAA01200N	Steering Wheel Assembly	1	

Item	Part Number	Description	Qty	Remarks
8	0080680	Anti Squeak Steering Column	0	24 x 11 3/4")
19	SF0401012	Washer Plain M8X16X1.5XZN5	0	NS
26	1102AAA01490N	Steering Gear Assembly	1	

Table excerpt: 4 of 40 extracted rows shown. Saved as: output/MAHINDRA_THAR_steering_gear_and_mountings.json

12.4 Tata Indica — Intake and Exhaust Valve

Correctly preserved a repeated item number: item “3” appears three times for three Valve Guide size variants (+0.05, +0.10, +0.20), matching the catalogue's own numbering rather than being renumbered to 3, 4, 5. Full walkthrough and screenshots in Section 11.

Item	Part Number	Description	Qty
1	570305129907	Valve Inlet	8
3	570305159905	Valve Guide +0.05	16
3	570305159906	Valve Guide +0.10	16
3	570305159907	Valve Guide +0.20	16

Table excerpt: 4 of 12 extracted rows shown. Saved as: output/TATA_INDICA_intake_and_exhaust_valve.json

13. Libraries Used & Their Purpose

Library	Role in the Document Agent
langgraph	Defines and runs the eight-step workflow that drives the whole pipeline.
langchain-openai (ChatOpenAI)	Client used to call the LLM for structured BOM extraction.
langchain-google-genai	Alternate LLM client integration available in nodes.py.
PyMuPDF (fitz)	Fast whole-document PDF opening and per-page text extraction for the page index.
pdfplumber	Text and table extraction on the small set of pages found to be relevant.
pydantic	Defines the BOMItem/BOM schemas and validates the LLM's output.
python-dotenv	Loads API keys and settings from a local .env file (llm.py).
tabulate	Prints the final BOM as a readable table in the console for quick checking.
difflib (stdlib)	Powers the fuzzy-matching fallback in page retrieval.
re (stdlib)	Used for keyword scoring and text cleanup.
json / pathlib (stdlib)	Parsing the LLM's JSON response and handling file paths for saving output.

14. Testing Methodology

Testing was done end-to-end rather than as isolated unit tests, since most failure modes come from how steps interact rather than from any one function on its own. For each catalogue, the process was:

- Run the full agent via `DocumentAgent.invoke(pdf_path, query)` on a real catalogue and a plain-language query.
- Check `state["status"]` and `state["error"]` at the end to confirm the run reached "saved" without failing earlier.
- Check the matched pages against the actual catalogue to confirm the right assembly section, and only that section, was picked.
- Open the saved BOM JSON and check every row against the source page: item numbers, part numbers, descriptions, quantities, and remarks.
- Re-run the same catalogue and query to confirm the output is stable, since `temperature=0` is used on the LLM call.

This **manual check** was used because no ready-made “correct answer” dataset exists for these catalogues yet. It is also how the failure cases documented in Section 16 were found.

15. Challenges & Solutions

Challenge	Solution
Catalogues place multiple assemblies on the same or nearby pages, risking rows from the wrong assembly being pulled in.	A clear prompt rule tells the LLM to stop as soon as a different assembly title appears.
Some catalogue text has small variations or typos that break exact-phrase page matching.	Three-stage retrieval (exact → keyword → fuzzy match) so retrieval fails gracefully instead of breaking outright.
The LLM inconsistently returned item numbers as text or numbers, especially for values like “2-1”.	Every part's item field is converted to a string before schema validation.
The LLM sometimes added extra column codes (like effectivity letters) onto the description.	A dedicated rule in the prompt with a before/after example.
Quantity codes such as “RF” cannot be a number but should not be dropped.	The prompt sets quantity to 0 and keeps the code in remarks instead of guessing a number.
Extracted text sometimes has stray characters and extra whitespace that clutter descriptions.	<code>clean_text()</code> strips known noise and expands a fixed abbreviation list before the text reaches the LLM.
Making sure the LLM's JSON is well-formed before it is trusted or saved.	The prompt requires JSON only, and <code>validate_bom</code> runs full schema validation, failing clearly on any mismatch.

16. Limitations

The following limitations are noted honestly rather than glossed over:

- The rule of dropping the lowest matched page (assumed to be an index page) is a reasonable guess, not a guarantee — it can be wrong if the assembly genuinely starts on that page.
- There is no automatic retry if the LLM returns malformed JSON; a bad response currently causes a hard failure rather than a retry with a corrected prompt.

- There is no automated accuracy score yet — correctness is checked manually row by row (Section 14), which will not scale well as more catalogues are added.
- The agent runs in a single pass with no human review step before a BOM is saved as final.

17. Conclusion

The Document Agent completes the first step of the spare-parts pipeline: **turning a PDF catalogue into a clean, validated Bill of Materials, saved as a JSON file**, driven by a plain-language query with no manual page lookup needed. Its three-stage retrieval, rule-based prompt, and schema validation together produced correct, checked extractions across four different real catalogues covering tractor, SUV, and passenger-car assemblies.

This gives a solid, tested base for the rest of the pipeline. The BOM JSON schema defined here is exactly what the Inventory Agent will read once it is built, and the limitations listed above give a clear list of things to improve as more catalogues are added.

18. Future Work

The **next milestone is the Vision Agent**: a multimodal model that reads exploded-view diagrams directly and identifies parts and part numbers visually, to complement the Document Agent's text-based extraction for cases where key information is only clear in the diagram. After that, planned work includes:

- Vision Agent (next milestone) — extract parts from exploded-view diagrams and match them against the Document Agent's BOM for the same assembly.
- Inventory Agent — design the mock stock database and a lookup keyed on part_number against the BOM output.
- Decision Agent — decide the shortage threshold logic (fixed or per part category) once inventory data is available.
- Report Agent — generate a structured fulfilment report using the outputs of all agents.
- Document Agent improvements — an LLM retry step on JSON parse failure, an automated test set once a checked BOM dataset exists, and better handling of multi-column pages.
- The project's “creative agent” addition — to be planned once the core five-agent pipeline works end-to-end.